

Digital System Design

Ch 12. Embedding Special IPs in your own SoC

Instructor: Woojoo Lee

Ph.D., Associate Professor

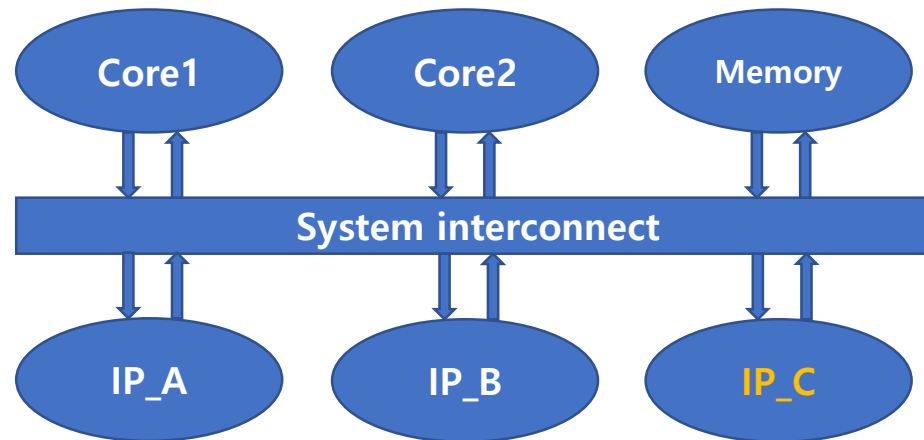
School of EEE, Chung-Ang University



SoC development

How to develop SoC?

- SoC for what? (let's focus on the embedded SoC)
- Why need to develop SoC? (From an industry perspective)
- General IP? Special IP?

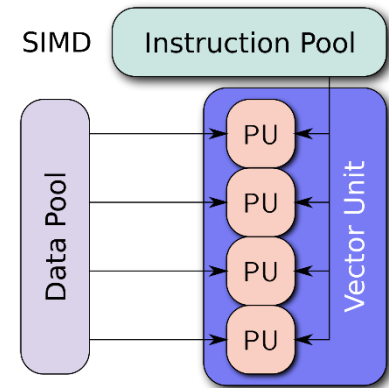


SoC development

Let's discuss more on the development of the special IPs.

- Example of the special IPs
 - DMA, SIMD, AI accelerator, etc.
- SIMD (single instruction multiple data)?
 - Let's implement the below example.

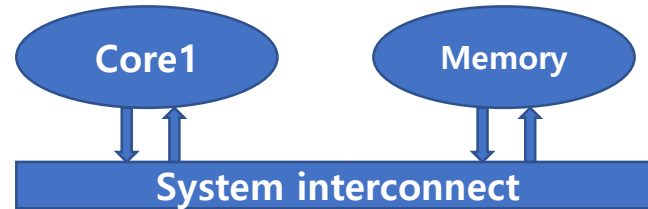
12	34	56	78
+	+	+	+
78	56	34	12
=	=	=	=
?	?	?	?



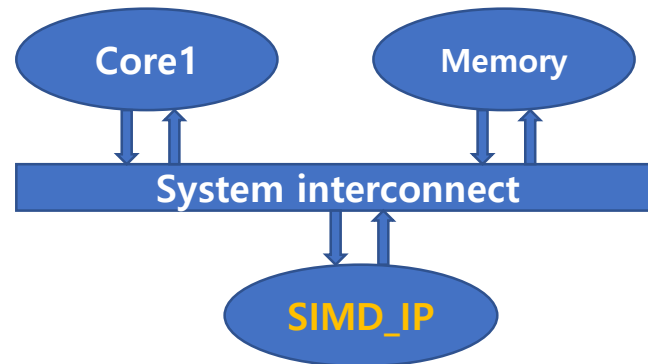
SoC development

(cont.)

- Using a single core..



- Developing a SIMD IP..



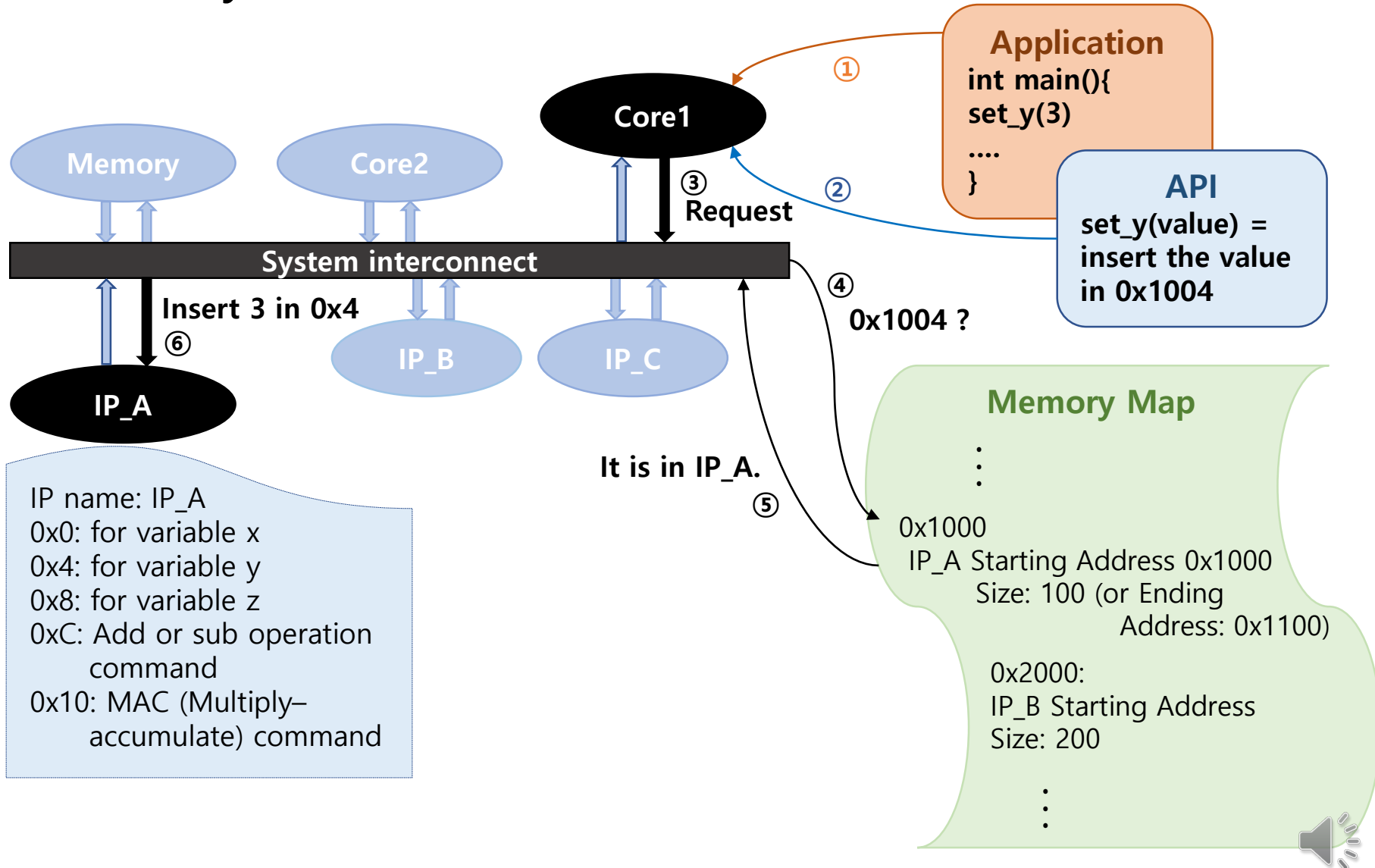
- Then, what do we need to develop the SIMD IP?

To answer this question, we need to know the SoC architecture.



SoC architecture

Let's briefly review the SoC architecture.

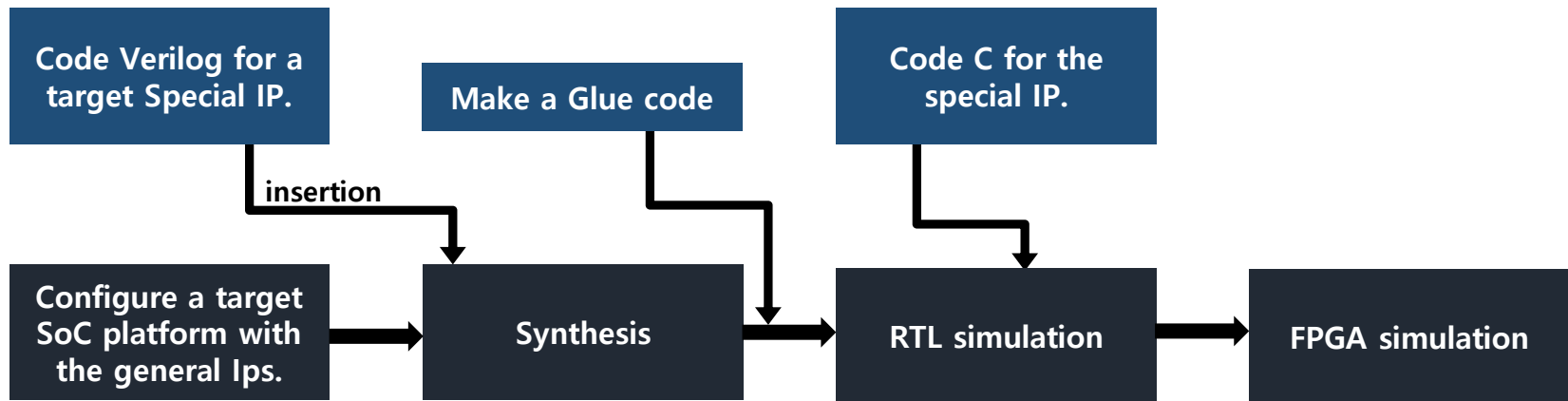


SoC development using RVX

Therefore, for developing a special IP, we need to do:

1. Signal processing HW for the on-chip communication (verilog coding)
2. HW logic for the special function (verilog coding)
3. Instruction API (C coding)

Okay, then why don't we study on how to develop a SoC with a special IP using RVX.



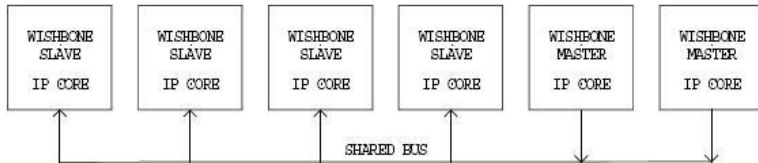
➤ Processes are **Outside RVX** and **In RVX**



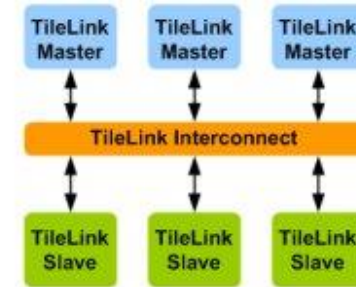
SoC development using RVX

Available communication architectures in the embedded SoC.

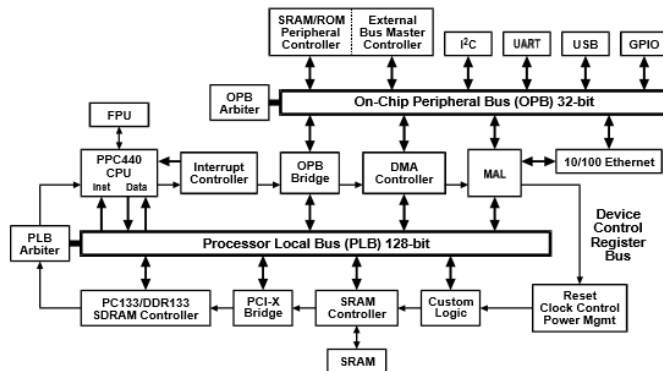
- Wishborn in OpenRISC



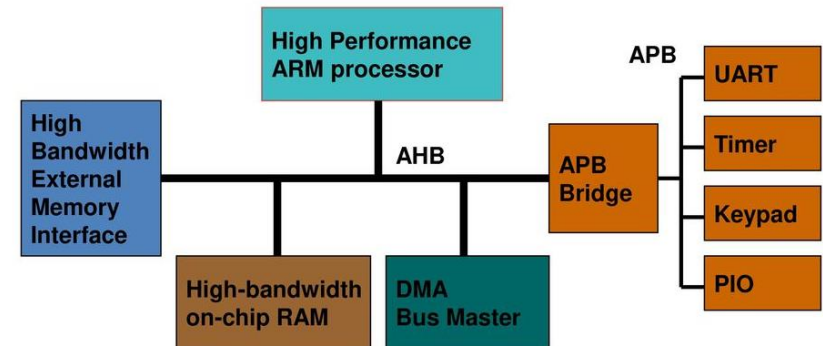
- Tilelink** from RISC-V



- CoreConnect from IBM



- AMBA (APB, AHB and AXI)** from ARM



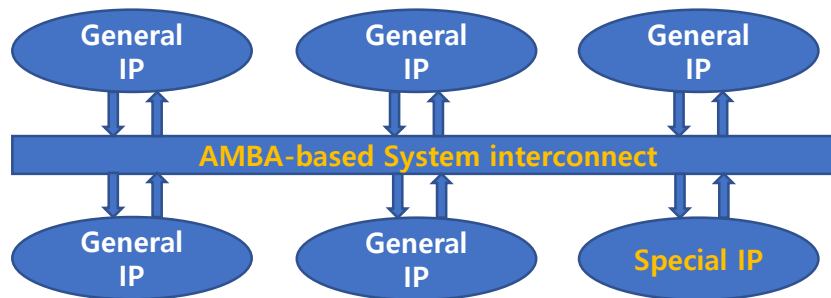
- OCP, SNP, XSNP, etc.



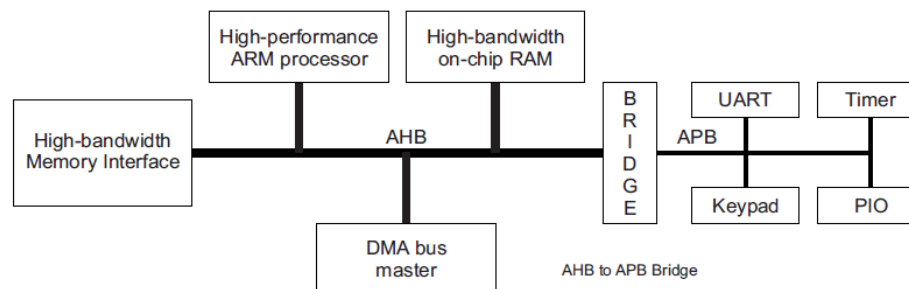
SoC development using RVX

Why use AMBA (Advanced Microcontroller Bus Architecture)?

- Most of the existing general IPs were developed based on the AMBA protocol, and these can be used when developing a new SoC.



※ RVX supports Tilelink to AMBA conversion to use RISC-V core.



- AMBA has APB, AHB, and AXI.

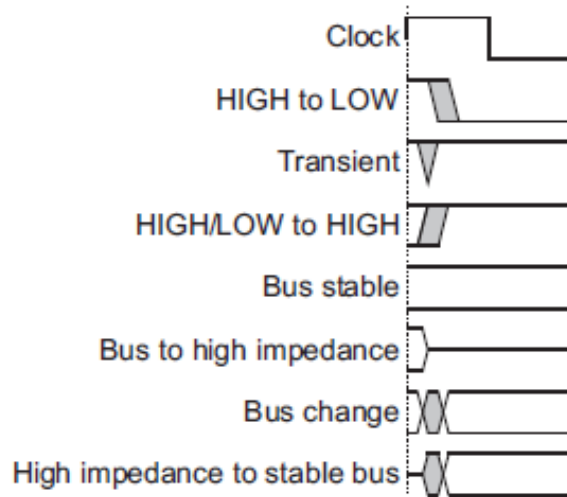


SoC development using RVX

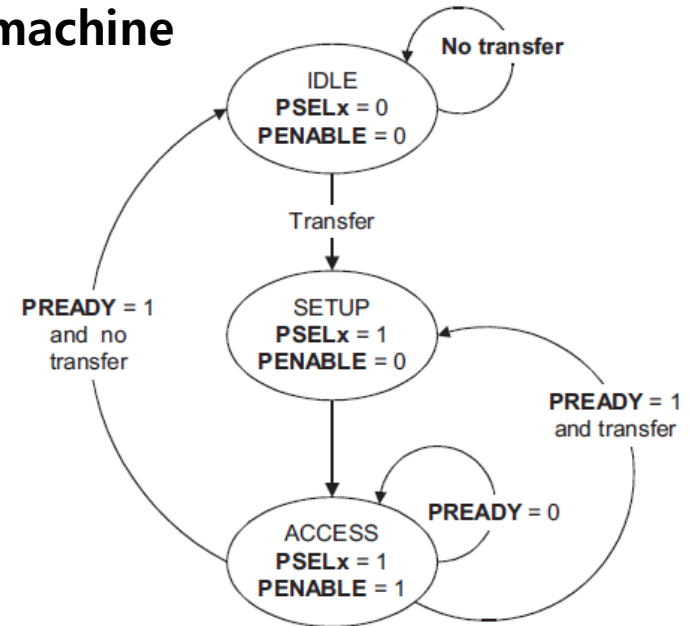
To develop a special IP (e.g., SIMD_IP), now we learn more about APB.

- APB is a simple bus.
 - Low performance
 - Single transfer
 - Suitable to control IPs
 - NOT allows partial write

Notation



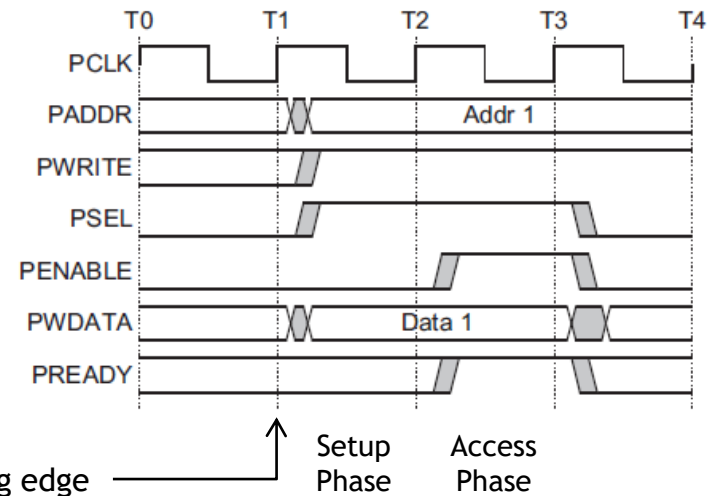
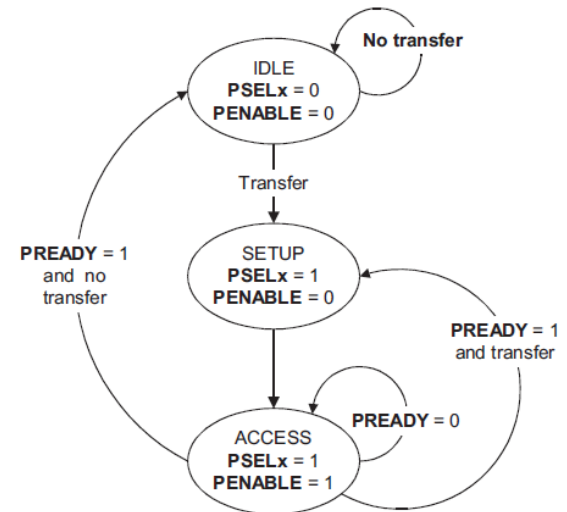
State machine



IP using APB

State machine of APB.

- **IDLE** state
 - Default APB state
- **SETUP** state
 - When transfer required
 - PSELx is asserted
 - Only one cycle
- **ACCESS** state
 - PENABLE is asserted
 - Addr, write, select, and write data remain stable
 - Stay if PREADY = L
 - Goto IDLE if PREADY = H and no more data
 - Goto SETUP if PREADY = H and more data pending



Starting with this rising edge



IP using APB

APB signal definition.

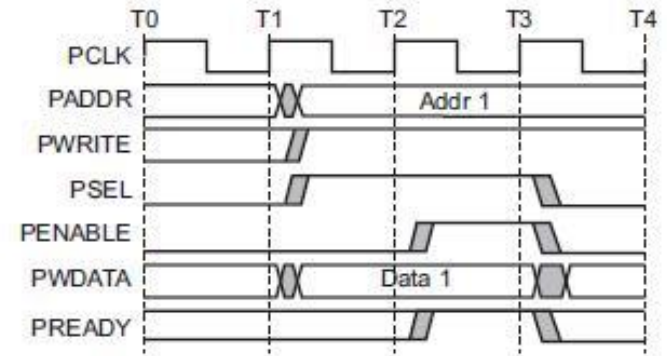
APB master IP: Core

Platform dependent signals

- PCLK: the bus clock source (rising-edge triggered)
- PRESETn: the bus (and typically system) reset signal (active low)
- PADDR: the APB address bus (can be up to 32-bits wide)
- PSELx: the select line for each slave device
- PENABLE: indicates the 2nd and subsequent cycles of an APB transfer
- PWRITE: indicates transfer direction (Write=H, Read=L)
- PWDATA: the write data bus (can be up to 32-bits wide)

APB slave IP: our special IP

- PREADY: used to extend a transfer
- PRDATA: the read data bus (can be up to 32-bits wide)
- PSLVERR: indicates a transfer error (OKAY=L, ERROR=H)



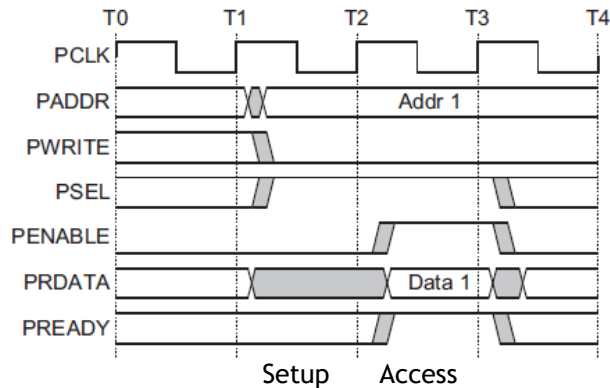
- **PCLK:** Clock
- **PADDR:** Address on bus
- **PWRITE:** 1=Write, 0=Read
- **PWDATA:** Data written to the I/O device
- **PSEL:** Asserted if the current bus transaction is targeted to *this* device
- **PENABLE:** High during entire transaction *other than* the first cycle.
- **PREADY:** Driven by target. Similar to our #ACK. Indicates if the target is *ready* to do transaction. Each target has it's own PREADY



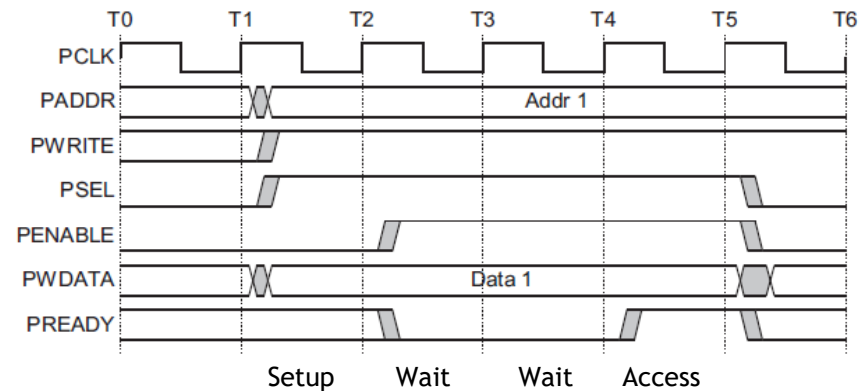
IP using APB

APB operation timing graph.

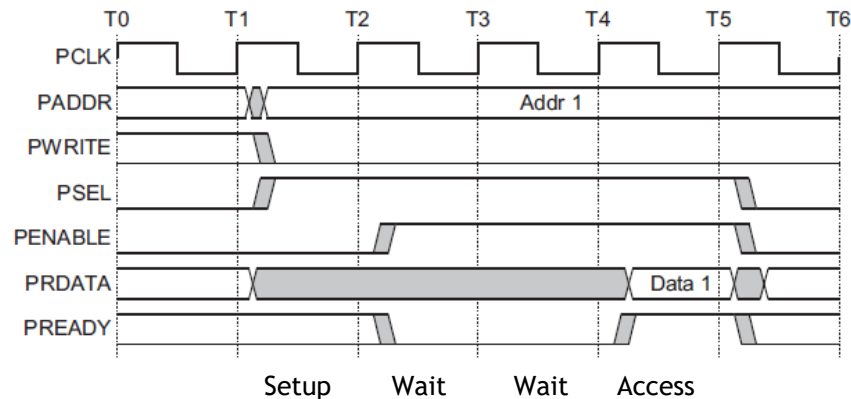
❖ A read transfer with no wait states



❖ A write transfer with wait states



❖ A read transfer with wait states



Development of Own SoC with a Special IP using RVX

Let's code verilog for a special IP.

1. Let's design an IP in "test1_apb.v".
2. The target IP has three registers for variable x, y, z, and support **add**, **sub** and **mac** commands.
 - 0x0: for variable x
 - 0x4: for variable y
 - 0x8: for variable z
 - 0xC: Add or sub operation command
 - 0x10: MAC (multiply and add) command



Development of Own SoC with a Special IP using RVX

Special IP RTL code – test1_apb.v

```
1 `include "ervp_global.vh"
2
3 module TEST1_APB
4 (
5     clk,
6     rstnn,
7
8     rpsel,
9     rpenable,
10    rpaddr,
11    rpwrite,
12    rpwdata,
13    rprdata,
14    rpready,
15    rpslverr
16 );
17
18 ///////////////////////////////////////////////////
19 /* parameter input output */
20 ///////////////////////////////////////////////////
21
22 parameter BW_ADDR = 1;
23 parameter BW_DATA = 1;
24
25 input wire clk, rstnn;
26 input wire rpsel;
27 input wire rpenable;
28 input wire [BW_ADDR-1:0] rpaddr;
29 input wire rpwrite;
30 input wire [BW_DATA-1:0] rpwdata;
31 output reg [BW_DATA-1:0] rprdata;
32 output wire rpready;
33 output reg rpslverr;
34
35 ///////////////
36 /* signals */
37 ///////////////
38
39 genvar i;
40
41 wire [8:1:0] addr_offset = rpaddr;
42 reg is_valid_addr;
43
44 reg [32:1:0] var_x, var_y, var_z;
45 reg we_x, we_y, we_z;
46
47 reg request_add;
48 reg request_sub;
49 reg request_mac;
50
51
52 always@(*)
53 begin
54     rprdata = 0;
55     is_valid_addr = 0;
56     we_x = 0;
57     we_y = 0;
58     we_z = 0;
59     request_add = 0;
60     request_sub = 0;
61     request_mac = 0;
62
63     if(rpsel==1'b 1)
64     begin
65         if(rpenable==1'b 1)
66         begin
67             case(addr_offset)
68             8'h 0:
69                 begin
70                     is_valid_addr = 1;
71                     rprdata = var_x;
72                     we_x = rpwrite;
73                 end
74             8'h 4:
75                 begin
76                     is_valid_addr = 1;
77                     rprdata = var_y;
78                     we_y = rpwrite;
79                 end
80             8'h 8:
81                 begin
82                     is_valid_addr = 1;
83                     rprdata = var_z;
84                     we_z = rpwrite;
85                 end
86             8'h C:
87                 begin
88                     is_valid_addr = 1;
89                     if(rpwrite==1'b 1)
90                         request_add = 1;
91                     else
92                         request_sub = 1;
93                 end
94             8'h 10:
95                 begin
96                     is_valid_addr = 1;
97                     if(rpwrite==1'b 1)
98                         request_mac = 1;
99                 end
100             default:
101                 is_valid_addr = 0;
102             endcase
103         end
104     end
105 end
106
107
108
109 assign rpready = 1;
110
111 always@(posedge clk, negedge rstnn)
112 begin
113     if(rstnn==0)
114         var_x <= 0;
115     else if(we_x==1'b 1)
116         var_x <= rpwdata;
117 end
118
119 always@(posedge clk, negedge rstnn)
120 begin
121     if(rstnn==0)
122         var_y <= 0;
123     else if(we_y==1'b 1)
124         var_y <= rpwdata;
125 end
126
127 always@(posedge clk, negedge rstnn)
128 begin
129     if(rstnn==0)
130         var_z <= 0;
131     else if(we_z==1'b 1)
132         var_z <= rpwdata;
133     else if(request_add)
134         var_z <= var_x + var_y;
135     else if(request_sub)
136         var_z <= var_x - var_y;
137     else if(request_mac)
138         var_z <= (var_x * var_y) + rpwdata;
139 end
140
141 always@(posedge clk, negedge rstnn)
142 begin
143     if(rstnn==0)
144         rpslverr <= 0;
145     else if(rpsel==1'b 1)
146     begin
147         if(rpenable==1'b 0)
148             rpslverr <= 0;
149         else
150             rpslverr <= ~is_valid_addr;
151     end
152 end
153
154 endmodule
155
156
```



Development of Own SoC with a Special IP using RVX

Let's add the code to the configured SoC platform.

- Take a look at the xml file for the platform configuration. Then add the "i_test1" as follows.

```
1  <?xml version="1.0" ?>
2  <rvx>
3    <platform>
4      <name>lec_apb</name>
5      <status>described</status>
6      <spec>
7        <define>
8          <name>sram_size</name>
9          <value>64kB</value>
10       </define>
11     </spec>
12     <ip_instance>
13       <name>i_main_core</name>
14       <library_name>rvc_orca</library_name>
15     </ip_instance>
16
17     <ip_instance>
18       <name>i_test1</name>
19       <library_name>user_slaveif_apb_clkin</library_name>
20       <parameter>
21         <id>SIZE_OF_MEMORYMAP</id>
22         <value>0x1000</value>
23       </parameter>
24       <parameter>
25         <id>BW_ADDR</id>
26         <value>32</value>
27       </parameter>
28       <parameter>
29         <id>BW_DATA</id>
30         <value>32</value>
31       </parameter>
32     </ip_instance>
33   </platform>
34 </rvx>
35
```

➡ SRAM size 설정

➡ RISC-V core instance

➡ IP를 APB slave로 설정

➡ 사용할 memory map size 설정

➡ address, data의 bandwidth 설정

"i_test1"이란 IP를
추가하는 부분



Development of Own SoC with a Special IP using RVX

Then, lets synthesize the platform, and perform the RTL simulation

- In RVX, command for synthesize is : make syn
- We need to manually connect the wires of the special IP and uNoC (system interconnection in RVX)

to complete the synthesis.

You can find this glue code in `lec_apb/user/rtl/include/tip_apb_user_region.vh`.

Edit the "tip_apb_user_region.vh." according to the previous "test1_apb.v".

We need to provide the tip_apb_user_region.vh code here!!



Development of Own SoC with a Special IP using RVX

Special IP Glue code – lec_apb_user_region.vh

```
1  /*****  
2  /* Custom Region */  
3  *****/  
4  
5  // wire clk_system;  
6  // wire clk_core;  
7  // wire clk_system_external;  
8  // wire clk_system_debug;  
9  // wire clk_interrupt;  
10 // wire clk_process_000;  
11 // wire clk_noc;  
12 // wire gclk_system;  
13 // wire gclk_core;  
14 // wire gclk_system_external;  
15 // wire gclk_system_debug;  
16 // wire gclk_interrupt;  
17 // wire gclk_process_000;  
18 // wire gclk_noc;  
19 // wire tick_1us;  
20 // wire tick_62d5ms;  
21 // wire global_rstnn;  
22 // wire global_rstpp;  
23 // wire [(5)-1:0] rstnn_sequence;  
24 // wire [(5)-1:0] rstpp_sequence;  
25 // wire rstnn_user;  
26 // wire rstpp_user;  
27 // wire i_test1_clk;  
28 // wire i_test1_rstnn;  
29 // wire i_test1_rpsel;  
30 // wire i_test1_rpenable;  
31 // wire i_test1_rpwwrite;  
32 // wire [(32)-1:0] i_test1_rpaddr;  
33 // wire [(32)-1:0] i_test1_rpdata;  
34 // wire i_test1_rpready;  
35 // wire [(32)-1:0] i_test1_rprdata;  
36 // wire i_test1_rpslverr;  
37  
38 /* DO NOT MODIFY THE ABOVE */  
39 /* MUST MODIFY THE BELOW */  
40  
41  
42 /*  
43 USER_IP  
44 #(  
45     .SIZE_OF_MEMORYMAP((32'h 1000)),  
46     .BW_ADDR(32),  
47     .BW_DATA(32)  
48 )  
49 i_test1  
50 (  
51     .clk(i_test1_clk),  
52     .rstnn(i_test1_rstnn),  
53     .rpsel(i_test1_rpsel),  
54     .rpenable(i_test1_rpenable),  
55     .rpwrite(i_test1_rpwwrite),  
56     .rpaddr(i_test1_rpaddr),  
57     .rpdata(i_test1_rpdata),  
58     .rpready(i_test1_rpready),  
59     .rprdata(i_test1_rprdata),  
60     .rpslverr(i_test1_rpslverr)  
61 );  
62 */  
63 //assign `NOT_CONNECT = i_test1_clk;  
64 //assign `NOT_CONNECT = i_test1_rstnn;  
65 //assign `NOT_CONNECT = i_test1_rpsel;  
66 //assign `NOT_CONNECT = i_test1_rpenable;  
67 //assign `NOT_CONNECT = i_test1_rpwwrite;  
68 //assign `NOT_CONNECT = i_test1_rpaddr;  
69 //assign `NOT_CONNECT = i_test1_rpdata;  
70 assign i_test1_rpready = 0;  
71 assign i_test1_rprdata = 0;  
72 assign i_test1_rpslverr = 0;  
73  
74
```

```
38 /* DO NOT MODIFY THE ABOVE */  
39 /* MUST MODIFY THE BELOW */  
40  
41  
42  
43 USER_IP  
44 #(  
45     .BW_ADDR(32),  
46     .BW_DATA(32)  
47 )  
48  
49 i_test1  
50 (  
51     .clk(i_test1_clk),  
52     .rstnn(i_test1_rstnn),  
53     .rpsel(i_test1_rpsel),  
54     .rpenable(i_test1_rpenable),  
55     .rpwrite(i_test1_rpwwrite),  
56     .rpaddr(i_test1_rpaddr),  
57     .rpdata(i_test1_rpdata),  
58     .rpready(i_test1_rpready),  
59     .rprdata(i_test1_rprdata),  
60     .rpslverr(i_test1_rpslverr)  
61 );  
62  
63 //assign `NOT_CONNECT = i_test1_clk;  
64 //assign `NOT_CONNECT = i_test1_rstnn;  
65 //assign `NOT_CONNECT = i_test1_rpsel;  
66 //assign `NOT_CONNECT = i_test1_rpenable;  
67 //assign `NOT_CONNECT = i_test1_rpwwrite;  
68 //assign `NOT_CONNECT = i_test1_rpaddr;  
69 //assign `NOT_CONNECT = i_test1_rpdata;  
70 //assign i_test1_rpready = 0;  
71 //assign i_test1_rprdata = 0;  
72 //assign i_test1_rpslverr = 0;  
73  
74
```



수정 전

수정 후



Development of Own SoC with a Special IP using RVX

Now, we need to develop a C code for the API of our special IP.

- Here is a sample code, saved in lec_apb/app/test_mmio/src/main.c

```
1 #include "platform_info.h"
2 #include "ervp_printf.h"
3 #include "ervp_reg_util.h"
4
5 #define OFFSET_X 0x0
6 #define OFFSET_Y 0x4
7 #define OFFSET_Z 0x8
8 #define OFFSET_ADD 0xC
9 #define OFFSET_SUB 0xC
10 #define OFFSET_MAX 0x10
11 #define OFFSET_NOI_VALID 0x3
12
13 static inline unsigned int get_test1_addr(unsigned int offset)
14 {
15     return (I_TEST1_SLAVE_BASEADDR+offset);
16 }
17
18 void set_x(unsigned int value)
19 {
20     REG32(get_test1_addr(OFFSET_X)) = value;
21 }
22
23 unsigned int get_x()
24 {
25     return REG32(get_test1_addr(OFFSET_X));
26 }
27
28 void set_y(unsigned int value)
29 {
30     REG32(get_test1_addr(OFFSET_Y)) = value;
31 }
32
33 unsigned int get_y()
34 {
35     return REG32(get_test1_addr(OFFSET_Y));
36 }
37
38 void set_z(unsigned int value)
39 {
40     REG32(get_test1_addr(OFFSET_Z)) = value;
41 }
42
43 unsigned int get_z()
44 {
45     return REG32(get_test1_addr(OFFSET_Z));
46 }
47
48 void perform_add()
49 {
50     REG32(get_test1_addr(OFFSET_ADD)) = 0;
51 }
52
53 void perform_sub()
54 {
55     int temp;
56     temp = REG32(get_test1_addr(OFFSET_SUB));
57 }
58
59 void perform_mac()
60 {
61     REG32(get_test1_addr(OFFSET_MAX)) = 3;
62 }
63
64 int main() {
65     set_x(3);
66     printf("\n%d", get_x());
67     set_y(6);
68     printf("\n%d", get_y());
69     set_z(4);
70     printf("\n%d", get_z());
71     perform_add();
72     printf("\n%d", get_z());
73     perform_sub();
74     printf("\n%d", get_z());
75     perform_mac();
76     printf("\n%d", get_z());
77     REG32(get_test1_addr(OFFSET_NOI_VALID)) = 0xffffffff;
78     return 0;
79 }
```

API

API

Application



Development of Own SoC with a Special IP using RVX

Finally, we are ready to perform the RTL simulation.

Platform spec : .../tip_apb/tip_apb.xml

RTL code : .../tip_apb/user/rtl/src/test1_apb.v

Glue code : .../tip_apb/user/rtl/include/tip_apb_user_region.vh

API code : .../tip_apb/app/test_mmio/src/main.c

After the RTL simulation, we can use RVX to do the FPGA prototyping.

- We have all experienced the FPGA prototyping and simulation, and we can do the same thing here.

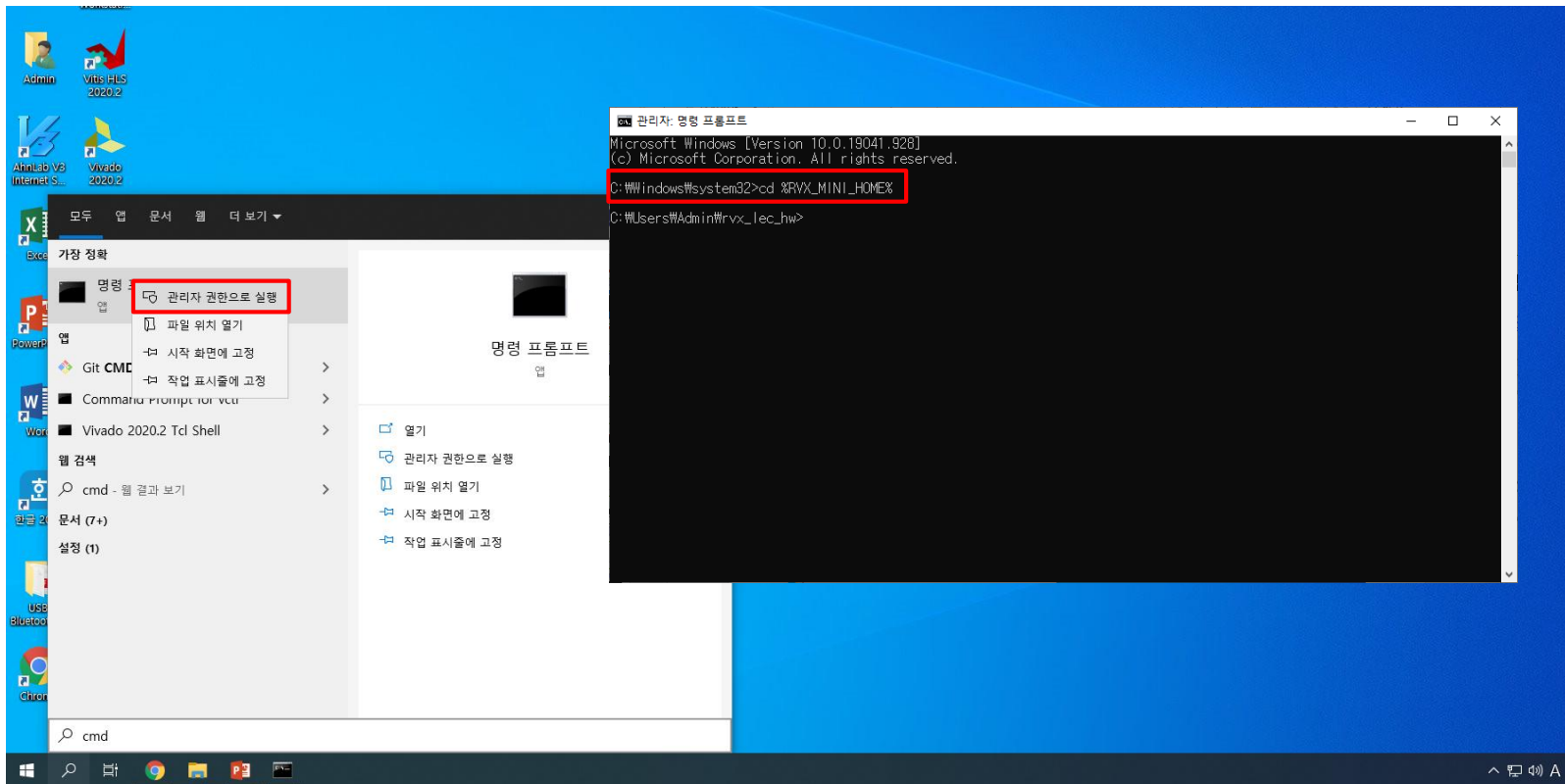


Practice

명령 프롬프트 실행

> cd %RVX_MINI_HOME%

1. 명령 프롬프트를 관리자 권한으로 실행
2. RVX_MINI_HOME으로 이동



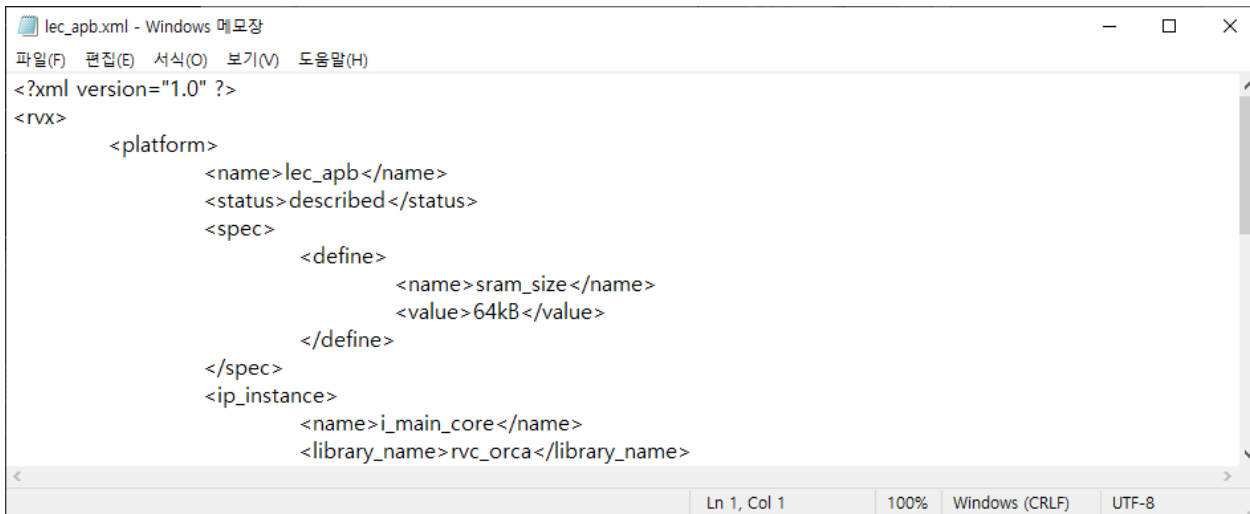
Practice

Special IP Platform Specification

- > cd platform\lec_apb
- > notepad lec_apb.xml

1. lec_apb platform 디렉토리로 이동
2. .xml 파일 open & 확인

```
C:\Windows\system32>cd %RVX_MINI_HOME%  
C:\Users\leegb\rvx\lec_hw>cd platform\lec_apb
```

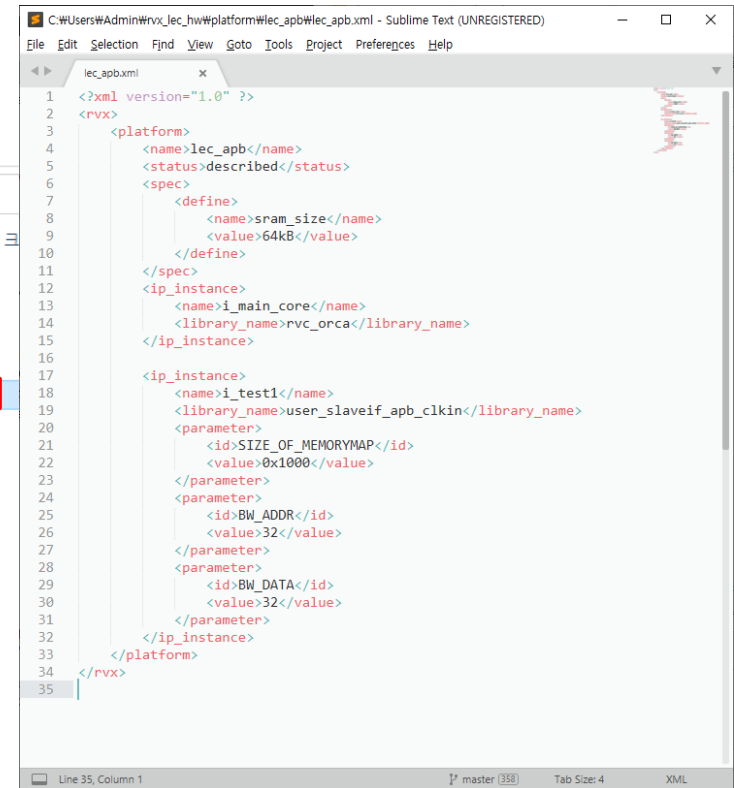
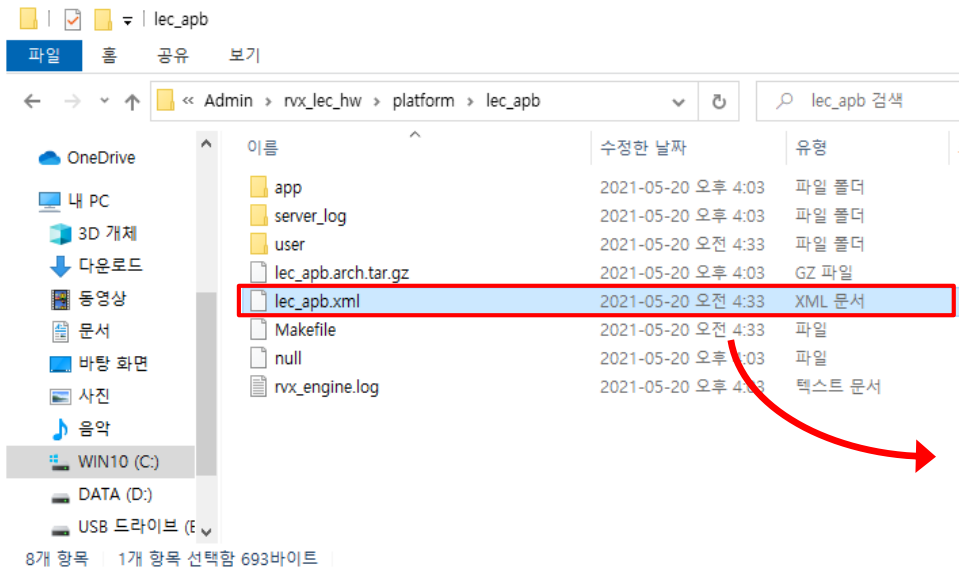


```
lec_apb.xml - Windows 메모장  
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)  
<?xml version="1.0" ?>  
<rvx>  
  <platform>  
    <name>lec_apb</name>  
    <status>described</status>  
    <spec>  
      <define>  
        <name>sram_size</name>  
        <value>64kB</value>  
      </define>  
    </spec>  
    <ip_instance>  
      <name>i_main_core</name>  
      <library_name>rvc_orca</library_name>  
    </ip_instance>  
  </platform>  
</rvx>
```

lec_apb는 custom IP를 만들어 APB로 연결한 platform
tip_artty와 마찬가지로 RVX에서 기본으로 제공 중



Special IP Platform Specification



파일탐색기에서 notepad가 아닌 다른 프로그램(web browser, text editor)으로 open 가능
(drag & drop 또는 연결프로그램 선택)



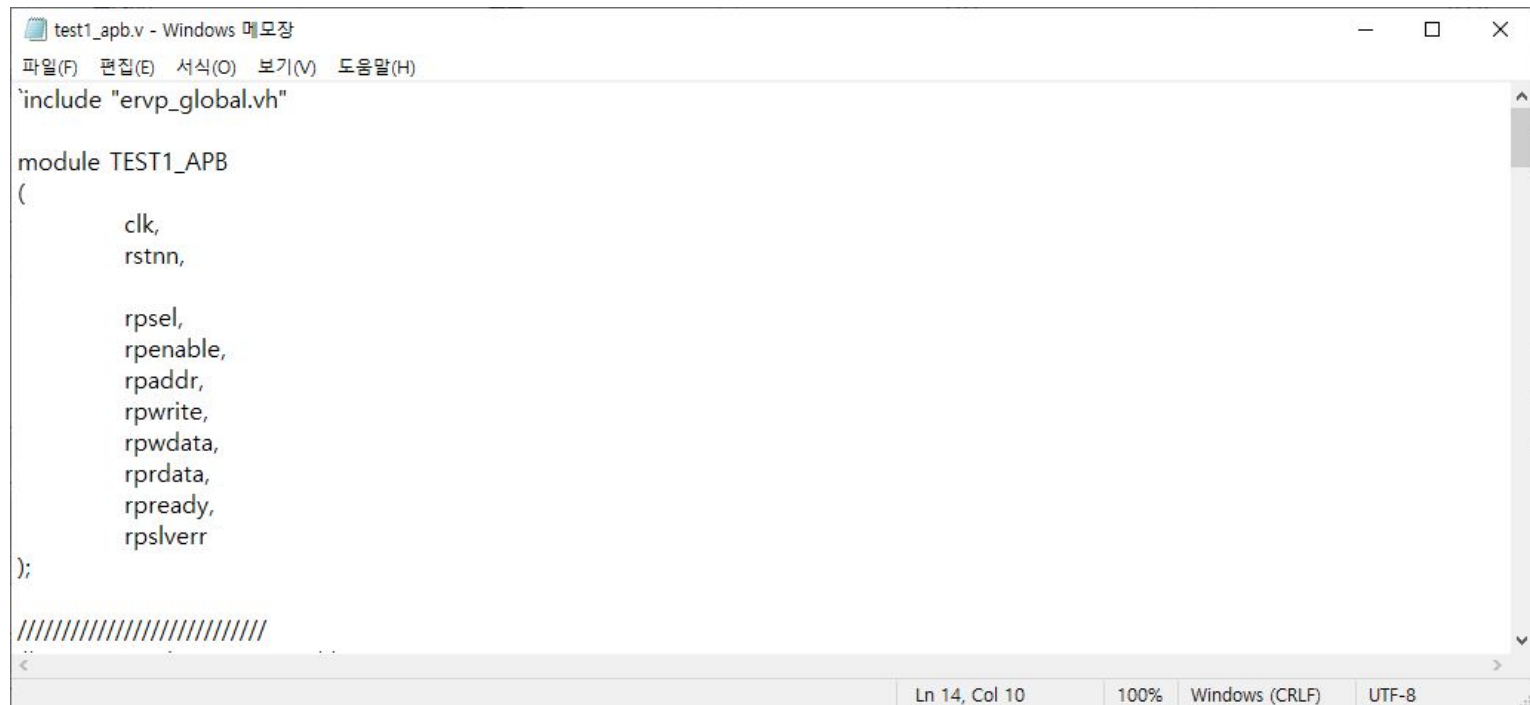
Practice

Special IP RTL code

1. ../user/rtl/src 디렉토리로 이동
2. test1_apb.v 소스코드 open & 확인

```
> cd user\rtl\src  
> notepad test1_apb.v
```

```
C:\Users\leegb\rvx_lec_hw\platform\lec_apb>cd user\rtl\src  
C:\Users\leegb\rvx_lec_hw\platform\lec_apb\user\rtl\src>notepad test1_apb.v
```



```
test1_apb.v - Windows 메모장  
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)  
`include "ervp_global.vh"  
  
module TEST1_APB  
(  
    clk,  
    rstnn,  
  
    rpsel,  
    rpenable,  
    rpaddr,  
    rpwrite,  
    rpwdata,  
    rprdata,  
    rpready,  
    rpslverr  
);  
  
////////////////////////////////////  
<
```



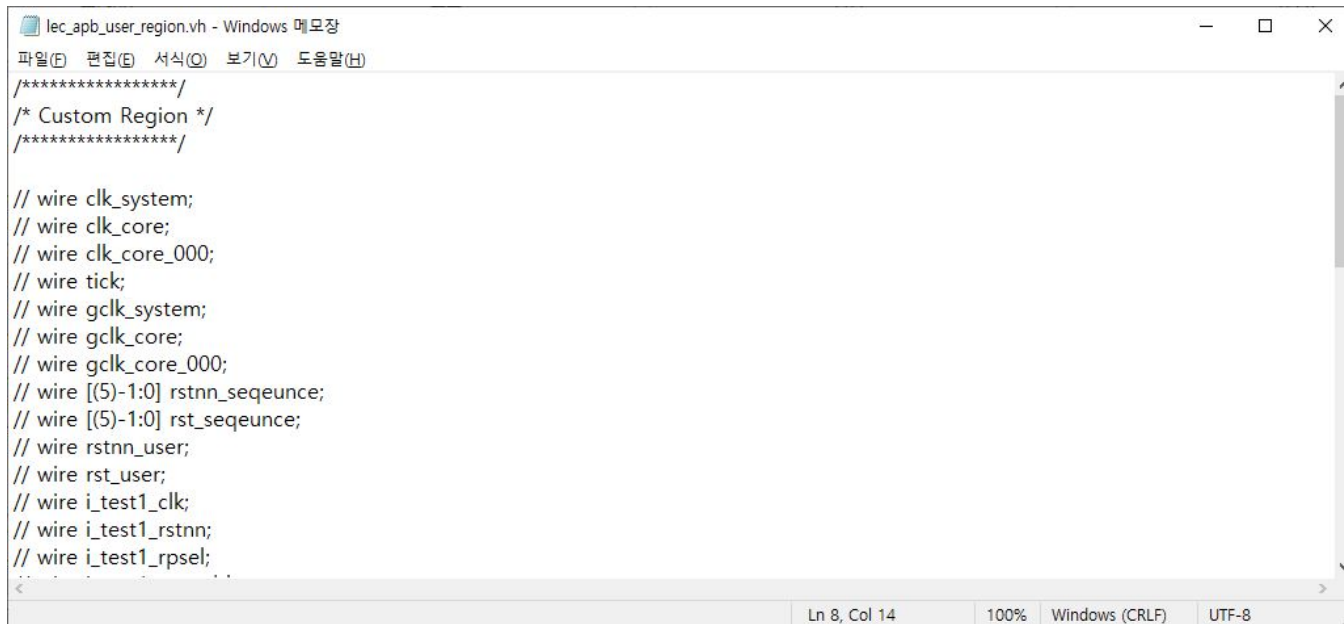
Practice

Special IP Glue code

1. ../user/rtl/include 디렉토리로 이동
2. test1_apb.v 소스코드 open & 확인

```
> cd ../include  
> notepad lec_apb_user_region.vh
```

```
C:\Users\leegb\rvx_lec_hw\platform\lec_apb\user\rtl\src>cd ../include  
C:\Users\leegb\rvx_lec_hw\platform\lec_apb\user\rtl\include>notepad lec_apb_user_region.vh
```



```
lec_apb_user_region.vh - Windows 메모장  
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)  
/*****  
/* Custom Region */  
*****/  
  
// wire clk_system;  
// wire clk_core;  
// wire clk_core_000;  
// wire tick;  
// wire gclk_system;  
// wire gclk_core;  
// wire gclk_core_000;  
// wire [(5)-1:0] rstnn_sequence;  
// wire [(5)-1:0] rst_sequence;  
// wire rstnn_user;  
// wire rst_user;  
// wire i_test1_clk;  
// wire i_test1_rstnn;  
// wire i_test1_rpsel;
```



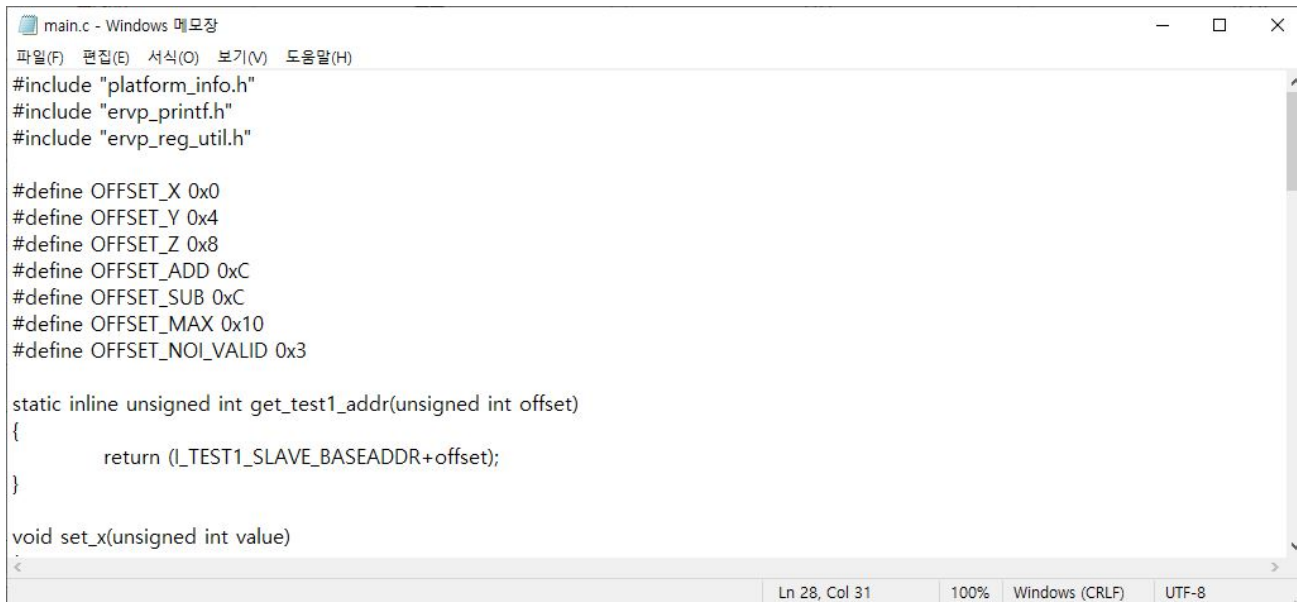
Practice

API & application code

```
> cd ../../app/test_mmio/src  
> notepad main.c
```

1. ../app/test_mmio/src 디렉토리로 이동
2. test1_apb.v 소스코드 open & 확인

```
C:\Users\leegb\rvx_lec_hw\platform\lec_apb\user\rtl\include>cd ../../app/test_mmio/src  
C:\Users\leegb\rvx_lec_hw\platform\lec_apb\app\test_mmio\src>notepad main.c
```



```
main.c - Windows 메모장  
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)  
#include "platform_info.h"  
#include "ervp_printf.h"  
#include "ervp_reg_util.h"  
  
#define OFFSET_X 0x0  
#define OFFSET_Y 0x4  
#define OFFSET_Z 0x8  
#define OFFSET_ADD 0xC  
#define OFFSET_SUB 0xC  
#define OFFSET_MAX 0x10  
#define OFFSET_NOI_VALID 0x3  
  
static inline unsigned int get_test1_addr(unsigned int offset)  
{  
    return (L_TEST1_SLAVE_BASEADDR+offset);  
}  
  
void set_x(unsigned int value)
```



Practice

Special IP Platform Synthesis

```
> cd ../../../../  
> make syn
```

1. lec_apb 디렉토리로 이동
2. Platform synthesis

```
C:\Users\leegb\rvx_lec_hw\platform\lec_apb\app\test_mmio\src>cd ../../../../  
C:\Users\leegb\rvx_lec_hw\platform\lec_apb>make syn
```

Special IP RTL simulation

```
> make sim_rtl  
> cd sim_rtl  
> make test_mmio
```

1. RTL simulation 디렉토리 생성
2. sim_rtl 디렉토리로 이동
3. test_mmio application으로 RTL sim.
(터미널 상으로 진행)

```
C:\Users\leegb\rvx_lec_hw\platform\lec_apb>make sim_rtl  
C:\Users\leegb\rvx_lec_hw\platform\lec_apb>cd sim_rtl  
C:\Users\leegb\rvx_lec_hw\platform\lec_apb\sim_rtl>make test_mmio
```



Special IP RTL simulation 출력 결과

```
# [JTAG: INFO] initialized
# [RESET CONTROLLER] all resets are released
# [JTAG: INFO] release all reset
# [JTAG: INFO] fast SRAM load start
# [JTAG: INFO] SRAM load end
# [JTAG: INFO] set sim env
# [JTAG: INFO] DR: 00000002
# [JTAG: INFO] set ready-to-run
# [JTAG: INFO] finished
#
# [SIM@RTL]
#
# 3
# 6
# 4
# 9
# -3
# 21
# [PROC_STATUS] 1 348348000
# ** Note: $finish : C:/Users/leegb/rvx_lec_hw/rvx_hwlib/lib_sim/ncsim/src/ncsim_manager.v(101)
# Time: 3596751640 ps Iteration: 0 Instance: /LEC_APB_SIM/i_ncsim_manager
# End time: 23:41:58 on May 20,2021, Elapsed time: 0:08:07
# Errors: 0, Warnings: 200
make[2]: Leaving directory `C:/Users/leegb/rvx_lec_hw/platform/lec_apb/sim_rtl'
make[1]: Leaving directory `C:/Users/leegb/rvx_lec_hw/platform/lec_apb/sim_rtl'
C:\Users\leegb\rvx_lec_hw\platform\lec_apb\sim_rtl>
```

make #(APP_NAME) 명령어로는 terminal 내부에서 simulation 진행
(수 분 소요)



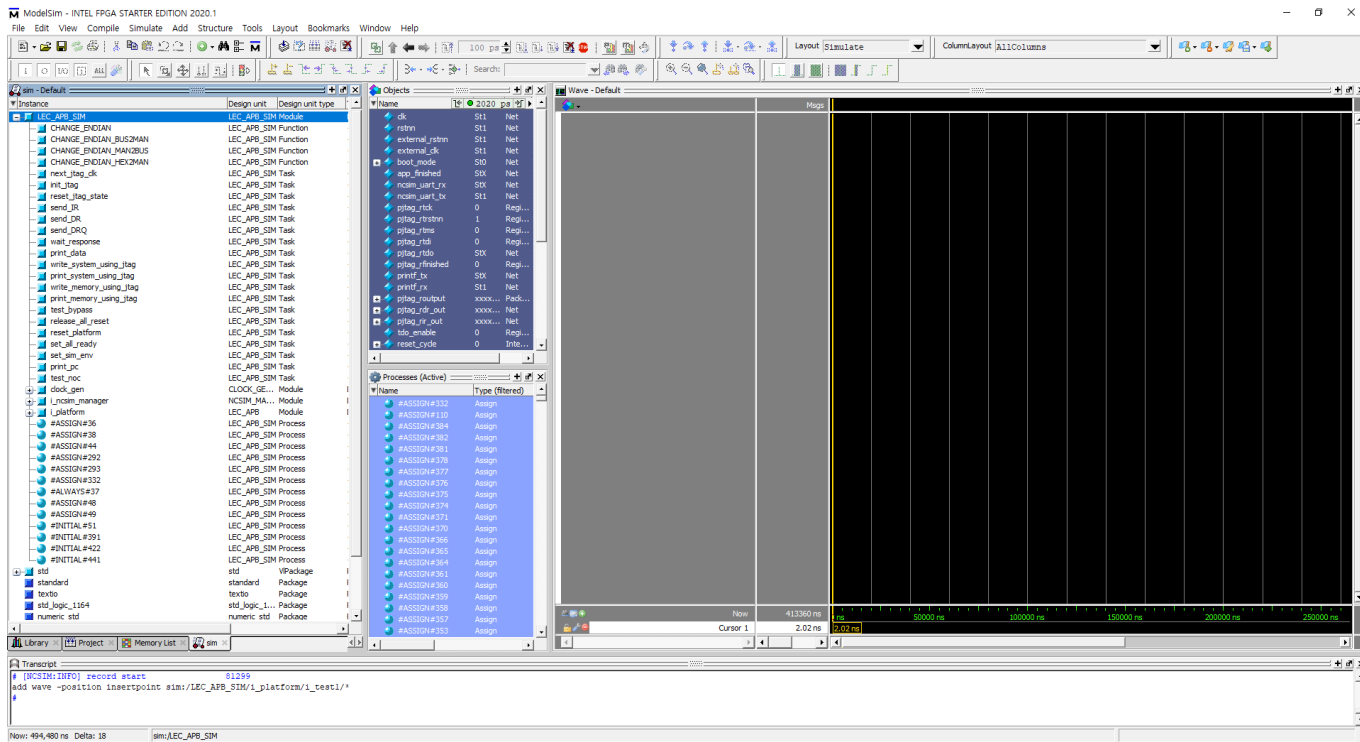
Practice

Special IP RTL simulation

1. Simulation GUI로 실행

> make test_mmio.debug_view

```
C:\Users\leegb\rvx_1ec_hw\platform\lec_apb\sim_rtl>make test_mmio.debug_view
```



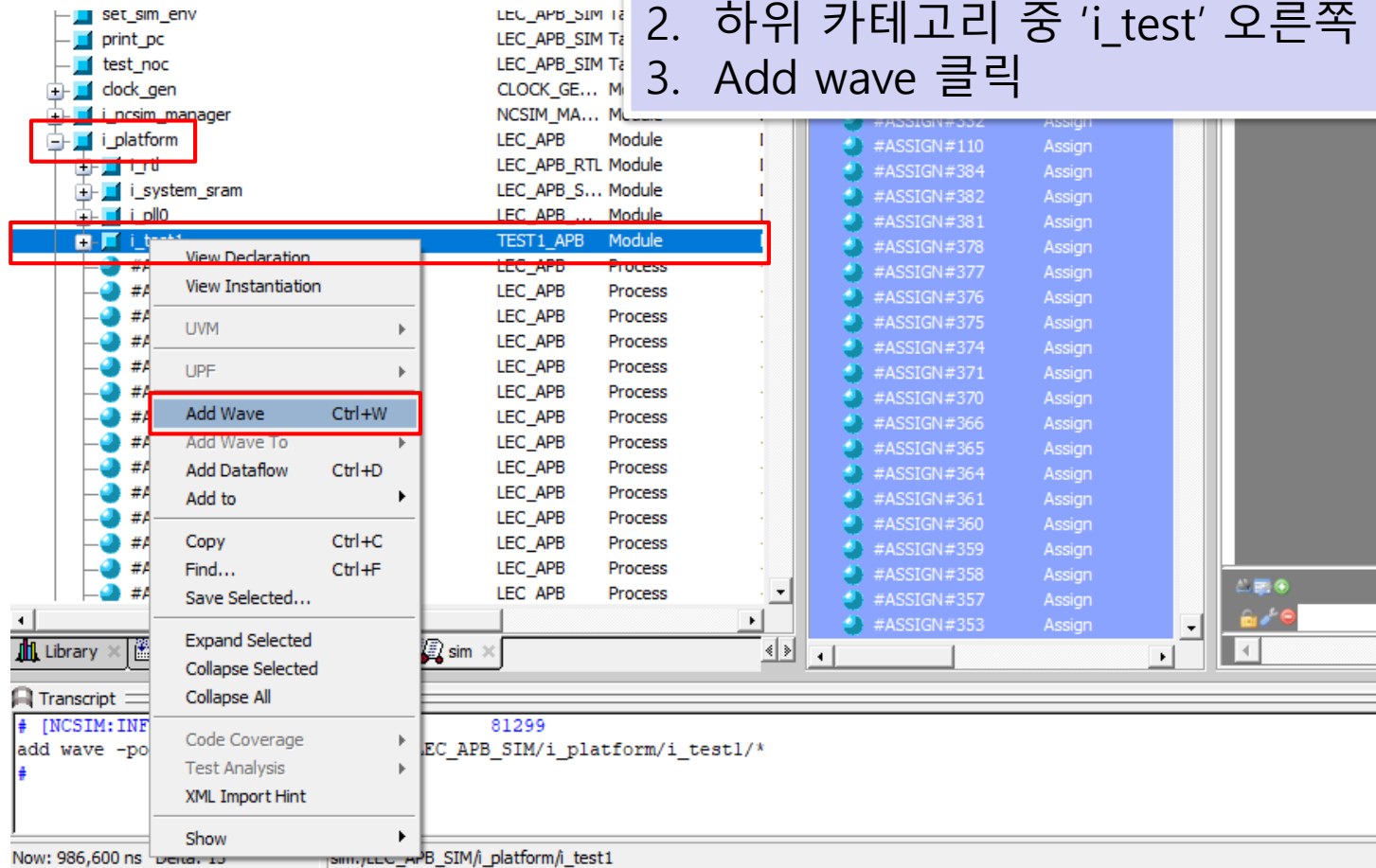
팝업 GUI로 modelsim이 실행되며, simulation이 완료되기까지 수 분 소요



Practice

Special IP signal wave 추가

1. i_platform 왼쪽 '+' 클릭
2. 하위 카테고리 중 'i_test' 오른쪽 클릭
3. Add wave 클릭



i_test 외에도 다른 IP의 wave를 추가하면 출력 가능



Special IP RTL simulation

Zoom Full (f)

1. 10진수로 변환할 variable 선택 후 오른쪽 클릭
2. Radix – Decimal 클릭
3. Simulation이 완료된 wave를 모두 보기 위해 Zoom Full 버튼 클릭 혹은 F 키 입력

위 스크린샷에서 편의상 터미널에 출력되는 사용자 변수를 10진수로 변환
이외에 address를 16진수로 변환하는 등 사용자 임의로 설정 가능



Error 해결 방법 – make syn

```
Elaborating is complete
Generating RTL is complete

Traceback (most recent call last):
  File "C:\\Users\\win0\\rvx_lec_hw\\env\\engine\\rvx_unified_engine.py", line 235, in <module>
    rvx_workspace.platform_syn(platform_name)
  File "C:\\Users\\win0\\rvx_lec_hw\\env\\engine\\rvx_workspace.py", line 732, in platform_syn
    self.__platform_syn_after(platform_name)
  File "C:\\Users\\win0\\rvx_lec_hw\\env\\engine\\rvx_workspace.py", line 716, in __platform_syn_after
    syn_user_file.rename(backup_user_file)
  File "C:\\Program Files\\WindowsApps\\PythonSoftwareFoundation.Python.3.9_3.9.1520.0_x64__qbz5n2kfra8p0\\Lib\\pathlib.py", line 1372, in rename
    self.accessor.rename(self, target)
FileExistsError: [WinError 183] 파일이 이미 있으므로 만들 수 없습니다: 'C:\\Users\\win0\\rvx_lec_hw\\platform\\lec_apb\\arch\\rtl\\include\\lec_apb_user_region.vh' -> 'C:\\Users\\win0\\rvx_lec_hw\\platform\\lec_apb\\user\\rtl\\include\\lec_apb_user_region.vh~'
make: *** [syn] 오류 1

C:\\Users\\win0\\rvx_lec_hw\\platform\\lec_apb>
```

Platform synthesis 과정(make syn)에서 위와 같은 오류가 발생하면,
.../lec_apb/user/rtl/include/lec_apb_user_region.vh~ 파일을 삭제한 후 다시 진행



Error 해결 방법 – make test_mmio.debug_view

```
*.o -march=rv32im -mabi=ilp32 -static -nostartfiles -lc -lm -lgcc -Wl,-gc-sections -o ./test_mmio.lec_apb.elf

1개 파일을 이동했습니다.
make[2]: Entering directory `C:/Users/win0/rvx_lec_hw/platform/lec_apb/sim_rtl'
process_begin: CreateProcess(NULL, rm -rf, ...) failed.
make (e=2): 지정된 파일을 찾을 수 없습니다.
make[2]: [__prepare_debug_rtl] 오류 2 (무시됨)
Model Technology ModelSim - Intel FPGA Edition vlog 2020.1 Compiler 2020.02 Feb 28 2020
Start time: 14:32:50 on May 21, 2021
vlog -work work -sv "+define+SIM_ENV" "+define+RTL_IMP" "+define+FAST_APP_LOAD" "+define+DISABLE_SPI_FLASH" "+define+RECORD_WAVE" "+
incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/munoc/include" "+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/peripheral/jtag_controller/incl
ude" "+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/peripheral/platform_controller/include" "+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib
/peripheral/common_peri_group/include" "+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/peripheral/uart/include" "+incdir+C:/Users/win0/r
vx_lec_hw/rvx_hwl/lib/peripheral/spi/include" "+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/peripheral/i2c/include" "+incdir+C:/Users/wi
n0/rvx_lec_hw/rvx_hwl/lib/peripheral/gpio/include" "+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/peripheral/external_peri_group/include"
"+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/core/orca_cache/include" "+incdir+C:/Users/win0/rvx_lec_hw/platform/lec_apb/arch/rtl/in
clude" "+incdir+C:/Users/win0/rvx_lec_hw/platform/lec_apb/user/rtl/include" "+incdir+C:/Users/win0/rvx_lec_hw/platform/lec_apb/user/
sim/include" "+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/lib_rtl/include" "+incdir+./include" "+incdir+C:/Users/win0/rvx_lec_hw/rvx
_hwl/lib/lib_sim/flash/include" "+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/lib_sim/memory/include" "+incdir+C:/Users/win0/rvx_lec_hw/r
vx_hwl/lib/lib_sim/misc/include" "+incdir+C:/Users/win0/rvx_lec_hw/rvx_hwl/lib/lib_sim/ncsim/include" "+incdir+C:/Users/win0/rvx_lec_hw/
rvx_hwl/lib/lib_sim/virtual_ip/include" C:/Users/win0/rvx_lec_hw/platform/lec_apb/arch/rtl/src/lec_apb.v C:/Users/win0/rvx_lec_hw/plat
form/lec_apb/arch/rtl/src/lec_apb_rtl.v C:/Users/win0/rvx_lec_hw/platform/lec_apb/user/rtl/src/test1_apb.v
Traceback (most recent call last):
  File "C:/Users/win0/rvx_lec_hw/rvx_util/simulate_rtl.py", line 112, in <module>
    execute_shell_cmd_separately(common_cmd_list, normal_verilog_file_list, args.each)
  File "C:/Users/win0/rvx_lec_hw/rvx_util/simulate_rtl.py", line 27, in execute_shell_cmd_separately
    execute_shell_cmd(cmd_list)
  File "C:/Users/win0/rvx_lec_hw/rvx_util/simulate_rtl.py", line 15, in execute_shell_cmd
    assert exit_code==0, cmd_list[-1]
AssertionError: C:/Users/win0/rvx_lec_hw/platform/lec_apb/user/rtl/src/test1_apb.v
make[2]: *** [__compile_rtl] 오류 1
make[2]: Leaving directory `C:/Users/win0/rvx_lec_hw/platform/lec_apb/sim_rtl'
make[2]: Entering directory `C:/Users/win0/rvx_lec_hw/platform/lec_apb/sim_rtl'
make[2]: Leaving directory `C:/Users/win0/rvx_lec_hw/platform/lec_apb/sim_rtl'
make[1]: Leaving directory `C:/Users/win0/rvx_lec_hw/platform/lec_apb/sim_rtl'

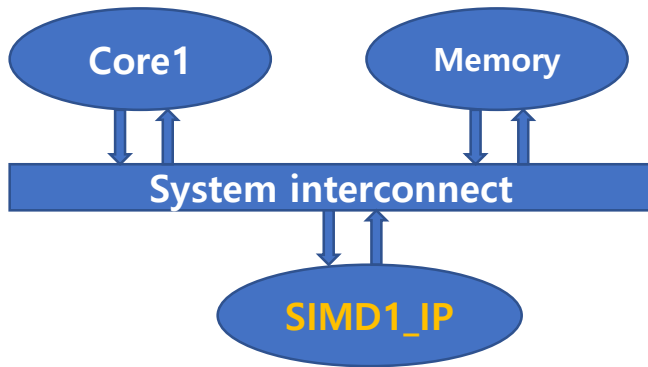
C:/Users/win0/rvx_lec_hw/platform/lec_apb/sim_rtl>_
```

RTL simulation 중 make test_mmio.debug_view 과정에서 위와 같은 오류가 발생하면,
다른 조치 없이 명령어를 2~3차례 다시 입력하면 오류 없이 진행됨.
파란색 부분은 무시되는 오류로 simulation에 영향이 없음.



Mini Project #1

SIMD IP 설계



12	34	56	78
+	+	+	+
98	76	54	32
=	=	=	=
?	?	?	?

1. RTL 코드를 수정하여 SIMD IP 설계 (코드 제출)
2. SIMD IP를 사용하기 위한 API (이름은 `perform_simd1`) 구현 (코드 제출)
3. 제시된 예제를 구현한 Application 코드 구현 (코드 제출)
4. 제시된 예제에 대한 시뮬레이션 및 검증 (waveform 스크린캡처 제출)

하나의 워드/한글 파일로 제출하기 바랍니다.

제출기한 : Eclass 강의게시판에 공지



Tip

SIMD IP 설계 – Hint!

예시) perform_simd 함수를 별도로 구현 후 결과를 z 변수에 하나로 묶어서 출력

test1_apb.v

```
always@(posedge clk, negedge rstnn)
begin
    if(rstnn==0)
        var_z <= 0;
    else if(we_z==1'b 1)
        var_z <= rpwdata;
    else if(request_add)
        var_z <= var_x + var_y;
    else if(request_sub)
        var_z <= var_x - var_y;
    else if(request_mac)
        var_z <= (var_x * var_y) + rpwdata;
    else if(request_simd)
        var_z <= var_simd;
end
```

```
always@(posedge clk, negedge rstnn)
begin
    if(rstnn==0)
        var_simd <= 0;
    else begin
        [redacted]
    end
end
```

➡ SIMD를 위한 logic

main.c

```
#define OFFSET_X 0x0
#define OFFSET_Y 0x4
#define OFFSET_Z 0x8
#define OFFSET_ADD 0xC
#define OFFSET_SUB 0xC
#define OFFSET_MAX 0x10
#define OFFSET_SIMD 0x14
#define OFFSET_NOI_VALID 0x3

void perform_simd()
{
    [redacted]
}
```

➡ SIMD를 위한 API

```
int main() {
    set_x(0x6745ABEF);
    printf("\n%x", get_x());
    set_y(0xABAB8967);
    printf("\n%x", get_y());
    perform_simd();
    printf("\n%x", get_z());
    return 0;
}
```

➡ 검증을 위한 app.



SIMD IP 설계 – Hint!

예시) perform_simd1 함수를 별도로 구현 결과를 z 변수에 하나로 묶어서 출력
– GUI simulation 중 확인해야하는 부분

